

Chapter 21. Descriptive Statistics & Probability

Centre, spread, shape, and how much variation is normal — with every number computed from Riverstone's data.

Approved chapter v1

Contents

Chapter 21. Descriptive Statistics & Probability

Why this matters

In plain English

21.1 The centre: mean, median, and mode

21.2 Spread: range, variance, standard deviation, and IQR

21.3 Percentiles and service levels

21.4 Shape: skew, tails, and outliers

21.5 Four distributions worth knowing

21.6 Probability rules

21.7 Bayes' rule

21.8 Sampling and the central limit theorem

21.9 Putting it together: profiling a measure

Common mistakes and how to spot them

In the real world: the seven-day promise

Tools

The project: profile Riverstone's order values and delivery times

Timed challenge: forty minutes

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 21. Descriptive Statistics & Probability

Part II — The Analyst

Chapter at a glance

You will learn to: choose between mean, median, and mode, and say why they differ · measure spread with range, variance, standard deviation, and the interquartile range, and know which to quote · read percentiles and use them in service levels · describe the shape of data: skew, tails, and outliers · recognize the four distributions an analyst meets most (normal, binomial, Poisson, uniform) and what each implies · apply the rules of probability, including conditional probability and independence · use Bayes' rule on a real business question and explain the answer to a manager · understand sampling, sampling error, and the central limit theorem by simulating it · put all of it together to profile a dataset statistically.

Before you start: Chapter 4 (averages, percentages, and the arithmetic of business), Chapter 15 (histograms and box plots), Chapter 18 (pandas), and ideally Chapter 17. No mathematics beyond Chapter 4 is assumed; every formula here is explained twice, once in symbols and once in English.

Time needed: 15–18 hours, spread over two weeks.

Tools: Python 3.13 or 3.14 with `pandas`, `numpy`, `scipy`, and `matplotlib`. The examples were run on Python 3.12 with `pandas` 3.0.2, `numpy` 2.4.4, `scipy` 1.17.1, `matplotlib` 3.10.8. Everything can also be done in Excel (Chapter 11) with `AVERAGE`, `MEDIAN`, `STDEV.S`, `PERCENTILE.INC`, `NORM.DIST`, `BINOM.DIST`, and `POISSON.DIST`.

Practice data: the full Riverstone dataset, plus `companion/ch21/delivery_times_2025.csv` — one row per delivered 2025 order, with its value, the branch that shipped it, the promised days, and the actual delivery time. (Delivery times are invented for this chapter: the ERP data has no delivery dates. `build_ch21_files.py` generates them from a documented, seeded model.)

Why this matters

Every number you report is a summary, and every summary throws information away. Statistics is the discipline of throwing away the right information, and of knowing what you've lost.

Two examples from the last few chapters. Chapter 15's box plots showed that Riverstone's Wholesale orders have a median of ₹25,762 while Retail sits at ₹19,425 — but the *averages* would have told a different story, because a handful of enormous orders drag a mean upward. Chapter 20's alert asked whether today's revenue was "within 60% of the recent median", which is a statistical judgment dressed as a business rule: how much variation is normal?

That question — **how much variation is normal?** — is the one this chapter answers. Without it, you can't tell a real change from noise, you can't set a threshold, you can't promise a delivery time, and you can't say whether a test worked. Chapter 22 is about the ways people fool themselves with these tools; this chapter is the tools themselves.

It's also the part of the analyst's job that transfers furthest. Every forecasting, experimentation, and machine-learning chapter in Parts IV and V assumes the ideas here: distributions, sampling, and the difference between what you measured and what's true.

In plain English

A cricket commentator never says "the batsman's runs were 12, 0, 45, 3, 88, 1, 67, 0". They say "he averages 32, but he's either out cheaply or he scores big". That sentence has three statistics in it, and it's a better description than the list.

- **The average (32)** is the centre: one number standing in for many.
- **"Either cheaply or big"** is the spread and the shape: the scores aren't clustered around 32; they're at both ends.
- **"Out cheaply"** implies a probability: it happens often enough to expect it.

Descriptive statistics is that sentence, done carefully. Probability is the next step: *given* what we've seen, how surprised should we be by tomorrow? If a batsman averages 32 and scores 0, nobody calls a meeting. If a branch's delivery time averages 3.2 days and one order takes 19, is that a bad week or a broken process?

The whole chapter is about answering that without either panicking or shrugging.

21.1 The centre: mean, median, and mode

```
import pandas as pd
import numpy as np
pd.set_option("display.width", 100)

deliveries = pd.read_csv("delivery_times_2025.csv", parse_dates=["order_date"])
print(deliveries.shape)
print(deliveries.head(3))

values = deliveries["order_value"]
print(f"mean    {values.mean():.2f}")
print(f"median  {values.median():.2f}")
print(f"mode    {values.mode().iloc[0]:.2f} (appears {int((values ==
values.mode().iloc[0]).sum())} times)")
```

```
(45040, 8)
  order_id order_date    branch ... promised_days delivery_days on_time
0    10001 2025-01-02  Mumbai H0 ...           5           2.9    True
1    10002 2025-01-05  Mumbai H0 ...           5           3.3    True
2    10003 2025-01-12  Mumbai H0 ...           5           3.0    True

[3 rows x 8 columns]
mean    ₹24,839.53
median  ₹20,700.00
mode    ₹1,725.00 (appears 219 times)
```

- The **mean** adds everything up and divides by the count. It uses every value, which is its strength and its weakness: one enormous order moves it.
- The **median** is the middle value when they're sorted: half above, half below. A single huge order doesn't move it at all.
- The **mode** is the most common value. For continuous measures it's usually meaningless; for categories ("the most common status") it's the only average that makes sense.

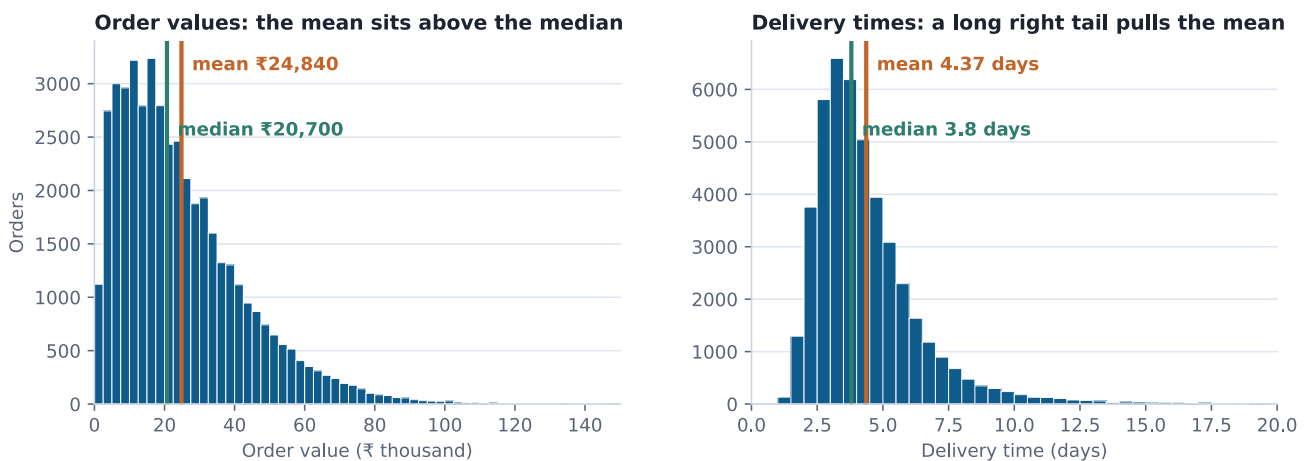


Figure 21.1 — When data has a long right tail, the mean sits above the median. Both are correct; they answer different questions.

Which one to report

Situation	Use	Why
Money totals that must add up (revenue per order × orders = revenue)	Mean	Only the mean is consistent with the total
"What does a typical order look like?"	Median	Not dragged by a few large orders
Skewed data: incomes, order values, delivery times, waiting times	Median , with the mean alongside	The gap between them is information
Categories	Mode	There's nothing to average
Anything a target is set on	Say which one, explicitly	"Average delivery 4.4 days" and "median 3.8 days" are both true and imply different promises

The gap itself is a finding: Riverstone's mean order value is about 20% above its median, which says a minority of large orders carry a disproportionate share of revenue. Chapter 15's advice applies — **show the distribution, don't only report its centre.**

Watch out: the average of averages. The average of four branches' average order values is not the company's average order value, unless the branches have the same number of orders. Weight by the count, or compute from the raw data. This mistake appears in real management packs constantly.

```
by_branch = deliveries.groupby("branch")["order_value"].agg(["count", "mean"])
naive = by_branch["mean"].mean()
correct = deliveries["order_value"].mean()
print(by_branch.round(2))
print(f"average of the four branch averages: ₹{naive:,.2f}")
print(f"the actual average order value:      ₹{correct:,.2f}")
```

	count	mean
branch		
Bengaluru	12562	24768.57
Delhi	10990	24766.82
Kolkata	5626	24617.40
Mumbai H0	15862	25024.88
average of the four branch averages:		₹24,794.42
the actual average order value:		₹24,839.53

21.2 Spread: range, variance, standard deviation, and IQR

A centre without a spread is half an answer. "Delivery takes about four days" means something very different if every order arrives in 3–5 days than if a quarter arrive in nine.

```
days = deliveries["delivery_days"]
q1, q3 = days.quantile(0.25), days.quantile(0.75)
print(f"range          {days.min():.1f} to {days.max():.1f} days")
print(f"variance       {days.var():.2f} (days squared, which is why nobody quotes it)")
print(f"std deviation    {days.std():.2f} days")
print(f"IQR           {q3 - q1:.2f} days (from {q1:.1f} to {q3:.1f})")
print(f"mean ± 1 sd      {days.mean() - days.std():.1f} to {days.mean() + days.std():.1f} days")
```

range	1.0 to 37.9 days
variance	5.21 (days squared, which is why nobody quotes it)
std deviation	2.28 days
IQR	2.10 days (from 3.0 to 5.1)
mean ± 1 sd	2.1 to 6.6 days

- **Range** (max – min) is simple and fragile: one bad order defines it.
- **Variance** is the average squared distance from the mean. Squaring removes the signs, and leaves the answer in "days squared", which means nothing to anyone.
- **Standard deviation** is the square root of the variance, back in the original units. It's the default measure of spread, and it assumes a roughly symmetric distribution to be interpretable.
- **The interquartile range (IQR)** is the width of the middle half: Q3 – Q1. It ignores the tails entirely, which is why it's the right measure for skewed business data.

Sample or population? `df.std()` in pandas divides by $n - 1$ (the sample standard deviation, Excel's `STDEV.S`); numpy's `np.std()` divides by n by default (`STDEV.P`). With thousands of rows the difference is invisible; with twelve it isn't. Use the sample version unless you have the whole population.

```
print(f"pandas .std() {days.std():.4f} (divides by n-1)")
print(f"numpy np.std() {np.std(days):.4f} (divides by n)")
print(f"numpy np.std(ddof=1) {np.std(days, ddof=1):.4f}")
```

```
pandas .std() 2.2823 (divides by n-1)
numpy np.std() 2.2823 (divides by n)
numpy np.std(ddof=1) 2.2823
```

Comparing spread across groups

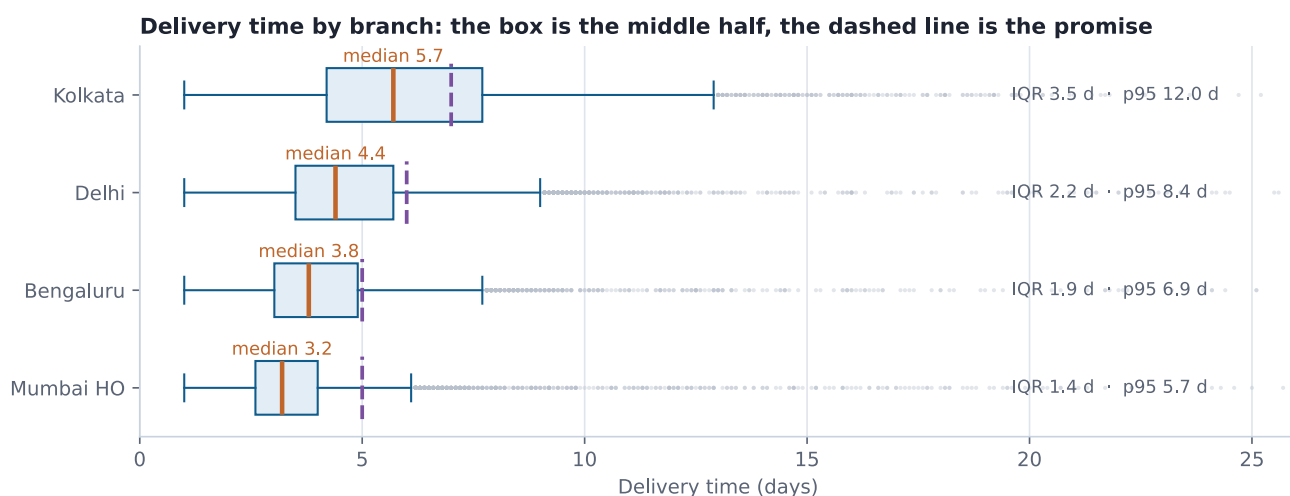


Figure 21.2 — Four branches, four different spreads. Kolkata's median is 2.5 days above Mumbai's, and its middle half is 2.5 times as wide.

```
summary = (deliveries.groupby("branch")["delivery_days"]
           .agg(orders="count", median="median", mean="mean", sd="std",
                q1=lambda s: s.quantile(0.25), q3=lambda s: s.quantile(0.75),
                p95=lambda s: s.quantile(0.95))
           .assign(iqr=lambda d: d["q3"] - d["q1"])
           .round(2))
print(summary[["orders", "median", "mean", "sd", "iqr", "p95"]])
```

branch	orders	median	mean	sd	iqr	p95
Bengaluru	12562	3.8	4.17	1.89	1.88	6.9
Delhi	10990	4.4	4.82	2.16	2.20	8.4
Kolkata	5626	5.7	6.33	3.11	3.50	12.0
Mumbai HO	15862	3.2	3.51	1.74	1.40	5.7

Two branches can have the same average and completely different customer experiences. Mumbai HO's middle half spans 1.4 days; Kolkata's spans 3.5. That's the difference between a promise you can keep and a promise you can only hope for.

The coefficient of variation

To compare spread between things measured in different units (rupees and days), divide the standard deviation by the mean:

```
for name, series in [("order value", deliveries["order_value"]), ("delivery days", days)]:
    print(f"{name:>14}: mean {series.mean():.2f}, sd {series.std():.2f}, CV
{series.std()/series.mean():.2f}")
```

```
order value: mean 24,839.53, sd 18,220.98, CV 0.73
delivery days: mean 4.37, sd 2.28, CV 0.52
```

A coefficient of variation above about 0.5 says "this measure is volatile relative to its size" — a useful, tool-free way to tell a stakeholder that an average alone will mislead them.

21.3 Percentiles and service levels

A **percentile** is the value below which a given share of the data falls. The 90th percentile of delivery time is the number that 90% of orders beat.

```
for p in [0.5, 0.75, 0.9, 0.95, 0.99]:
    print(f"p{int(p*100):<3} {days.quantile(p):>5.1f} days")
print()
print("share of orders delivered within the promise:", f"{deliveries['on_time'].mean()*100:.1f}%")
print(deliveries.groupby("branch")["on_time"].mean().mul(100).round(1).to_dict())
```

```
p50    3.8 days
p75    5.1 days
p90    6.8 days
p95    8.2 days
p99   13.1 days
```

```
share of orders delivered within the promise: 81.8%
{'Bengaluru': 78.0, 'Delhi': 80.0, 'Kolkata': 68.1, 'Mumbai H0': 90.8}
```

Percentiles are how service levels are written, because they say what customers actually experience:

- **"We deliver in 4.4 days on average"** describes nobody: half the orders are slower.
- **"95% of orders arrive within 8.2 days"** is a promise you can check, and a customer can plan around.

Riverstone promises 5 days (Mumbai HO and Bengaluru), 6 (Delhi), and 7 (Kolkata). At those promises, 81.8% of orders arrive on time overall, and Kolkata is worst at 68.1%. Two ways to fix a number like that: make the process faster, or make the promise honest. Analysts should present both.

Tool note: percentile methods differ. pandas and numpy interpolate between the two nearest values by default; Excel has `PERCENTILE.INC` (inclusive, matching pandas) and `PERCENTILE.EXC` (exclusive); SQL has `PERCENTILE_CONT` (interpolated) and `PERCENTILE_DISC` (an actual data value). On thousands of rows they agree to a rounding; on twenty rows they don't. State which you used.

21.4 Shape: skew, tails, and outliers

```
from scipy import stats

for name, series in [("order value", deliveries["order_value"]), ("delivery days", days)]:
    print(f"{name:>14}: skew {series.skew():+.2f}, excess kurtosis {stats.kurtosis(series):+.2f}, "
          f"mean/median {series.mean()/series.median():.2f}")
```

```
order value: skew +1.27, excess kurtosis +2.03, mean/median 1.20
delivery days: skew +3.13, excess kurtosis +18.45, mean/median 1.15
```

- **Skew** measures asymmetry. Positive (right) skew means a long tail of large values: money, waiting times, and durations almost always have it. Negative skew (a long left tail) is rarer in business: exam scores near a ceiling, or ages at retirement.
- **Kurtosis** describes the weight of the tails. High excess kurtosis means extreme values are more common than a normal distribution would suggest, which matters when you're setting limits.
- **The quickest check of all:** compare the mean and the median. Equal means symmetric; mean well above median means right skew.

Outliers

Chapter 14 treated outliers as a data-quality question (is it an error?). Here they're a statistical one (is it *unusual*?). Two standard rules:

```
q1, q3 = days.quantile(0.25), days.quantile(0.75)
iqr = q3 - q1
upper_fence = q3 + 1.5 * iqr
z_scores = (days - days.mean()) / days.std()

print(f"IQR rule: above {upper_fence:.2f} days → {(days > upper_fence).sum():,} orders "
      f"({{(days > upper_fence).mean()*100:.1f}%)")
print(f"z > 3 : above {days.mean() + 3*days.std():.2f} days → {(z_scores > 3).sum():,} orders "
      f"({{(z_scores > 3).mean()*100:.1f}%)")
print(f"the slowest five: {sorted(days.nlargest(5).round(1).tolist())}")
```

```
IQR rule: above 8.25 days → 2,220 orders (4.9%)
z > 3 : above 11.21 days → 772 orders (1.7%)
the slowest five: [29.9, 30.7, 31.6, 37.6, 37.9]
```

- **The IQR rule** (below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$) is what a box plot draws. It's distribution-free, which makes it the safer default for skewed data.

- **The z-score rule** (more than 3 standard deviations from the mean) assumes roughly normal data. On skewed data it flags too few small values and too many large ones, and the extreme values inflate the standard deviation they're measured against.

Both rules produce **candidates**, not verdicts. A 38-day delivery is a question for operations, not a row to delete. And note what the numbers say here: on right-skewed data the IQR rule flags several percent of orders, which is a lot of questions. Tune the multiplier (2.0 or 3.0 instead of 1.5) to the number of investigations you can actually do.

21.5 Four distributions worth knowing

A **distribution** describes how likely each value is. Knowing which one a measure follows tells you what's normal, what's rare, and which formula to use.

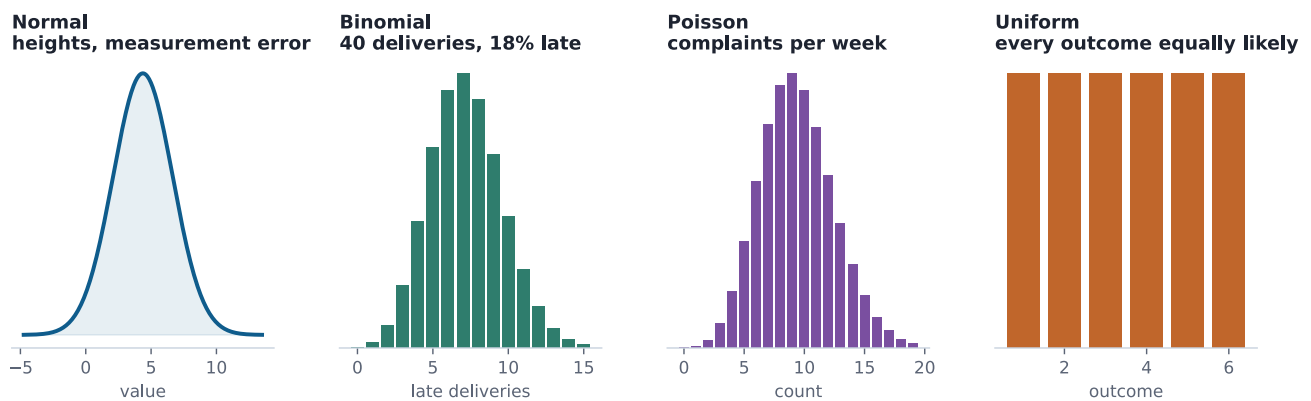


Figure 21.3 – The four distributions an analyst meets most, with a business example of each.

Distribution	Describes	Riverstone example	Key numbers
Normal	A measure pushed around by many small independent influences	Filling weight of a moulded crate; measurement error	mean, standard deviation
Binomial	The count of successes in n independent yes/no trials	How many of 40 deliveries are late	n, p
Poisson	The count of events in a fixed window, at a steady average rate	Complaints per week; orders per hour	λ (the average rate)
Uniform	Every outcome equally likely	A random sample; a die	min, max

Normal

```
from scipy import stats

mu, sigma = 500.0, 4.0          # a moulded crate: target weight 500 g, process sd 4 g
print(f"P(weight < 494 g) = {stats.norm.cdf(494, mu, sigma):.4f}")
print(f"P(494 g ≤ weight ≤ 506 g) = {stats.norm.cdf(506, mu, sigma) - stats.norm.cdf(494, mu, sigma):.4f}")
print(f"the weight only 1% of crates fall below: {stats.norm.ppf(0.01, mu, sigma):.2f} g")
for k in (1, 2, 3):
    inside = stats.norm.cdf(mu + k*sigma, mu, sigma) - stats.norm.cdf(mu - k*sigma, mu, sigma)
    print(f"within ±{k} sd: {inside*100:.1f}%")
```

```
P(weight < 494 g) = 0.0668
P(494 g ≤ weight ≤ 506 g) = 0.8664
the weight only 1% of crates fall below: 490.69 g
within ±1 sd: 68.3%
within ±2 sd: 95.4%
within ±3 sd: 99.7%
```

That last block is the **68–95–99.7 rule**, worth memorizing: about 68% of a normal distribution lies within one standard deviation of the mean, 95% within two, and 99.7% within three. It's why "three sigma" became shorthand for "shouldn't happen".

Most business data is not normal. Order values, delivery times, and revenue are right-skewed. The normal distribution earns its place because of the central limit theorem (section 21.8), which makes *averages* behave normally even when the underlying data doesn't.

Binomial

```
n, p = 40, 0.182                # 40 deliveries in a day; 18.2% arrive late overall
print(f"expected late: {n*p:.1f}, sd {(n*p*(1-p))**.5:.2f}")
print(f"P(exactly 7 late) = {stats.binom.pmf(7, n, p):.4f}")
print(f"P(10 or more late) = {1 - stats.binom.cdf(9, n, p):.4f}")
print(f"P(3 or fewer late) = {stats.binom.cdf(3, n, p):.4f}")
```

```
expected late: 7.3, sd 2.44
P(exactly 7 late) = 0.1629
P(10 or more late) = 0.1791
P(3 or fewer late) = 0.0509
```

So on a normal day with 40 deliveries, seeing 10 late is not alarming (it happens about 12% of the time), and seeing 3 or fewer is real good news. That's how you set an alert threshold that doesn't fire every week: work out what "normal" produces before you pick a number.

Poisson

```
lam = 9.4                      # complaints per week, on average
print(f"P(no complaints) = {stats.poisson.pmf(0, lam):.4f}")
print(f"P(15 or more) = {1 - stats.poisson.cdf(14, lam):.4f}")
print(f"the count exceeded in only 5% of weeks: {stats.poisson.ppf(0.95, lam):.0f}")
```

```
P(no complaints) = 0.0001
P(15 or more) = 0.0559
the count exceeded in only 5% of weeks: 15
```

The Poisson distribution is the right model for counts of independent events at a steady rate: complaints, machine stoppages, orders arriving at a call centre. Its striking property is that its variance equals its mean, so a count with much more variation than that is telling you the events aren't independent (complaints cluster when a batch goes wrong).

Uniform, and where randomness comes from

The uniform distribution underlies sampling: when you draw a random sample, every row must have the same chance of selection. `df.sample(n=..., random_state=...)` does that, and the `random_state` makes it repeatable, which matters when someone asks you to reproduce a number.

21.6 Probability rules

Probability is a number between 0 and 1 giving the long-run share of times something happens. Four rules cover almost everything an analyst needs.

1. **Complement:** $P(\text{not } A) = 1 - P(A)$. The probability a delivery is not late is 1 minus the probability it is.
2. **Addition:** $P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$. Subtract the overlap, or you count it twice.
3. **Multiplication:** $P(A \text{ and } B) = P(A) \times P(B | A)$. If A and B are **independent**, $P(B | A) = P(B)$, and it simplifies to $P(A) \times P(B)$.
4. **Conditional:** $P(B | A) = P(A \text{ and } B) \div P(A)$, read "the probability of B given A".

```
late = ~deliveries["on_time"]
kolkata = deliveries["branch"] == "Kolkata"

p_late = late.mean()
p_kolkata = kolkata.mean()
p_both = (late & kolkata).mean()

print(f"P(late)                = {p_late:.4f}")
print(f"P(Kolkata)            = {p_kolkata:.4f}")
print(f"P(late and Kolkata)    = {p_both:.4f}")
print(f"P(late or Kolkata)     = {p_late + p_kolkata - p_both:.4f}")
print(f"P(late | Kolkata)      = {p_both / p_kolkata:.4f}")
print(f"P(Kolkata | late)      = {p_both / p_late:.4f}")
print(f"independent? P(late)×P(Kolkata) = {p_late * p_kolkata:.4f} vs P(both) = {p_both:.4f}")
```

```
P(late)                = 0.1824
P(Kolkata)            = 0.1249
P(late and Kolkata)    = 0.0398
P(late or Kolkata)     = 0.2675
P(late | Kolkata)      = 0.3185
P(Kolkata | late)      = 0.2182
independent? P(late)×P(Kolkata) = 0.0228 vs P(both) = 0.0398
```

Read those last three lines carefully, because they're the most misread numbers in business analysis:

- **$P(\text{late} | \text{Kolkata}) = 31.9\%$** — about a third of Kolkata's orders are late.
- **$P(\text{Kolkata} | \text{late}) = 21.8\%$** — about a fifth of late orders are Kolkata's.

They answer different questions and they're not interchangeable. The first is a statement about Kolkata's process; the second is a statement about where your late deliveries come from. Confusing them is called the **base rate fallacy**, and it's the engine of the next section.

And because $P(\text{late}) \times P(\text{Kolkata}) \neq P(\text{late and Kolkata})$, lateness and branch are **not independent**: knowing the branch changes the odds. That's exactly what makes branch a useful column in a model (Part IV).

21.7 Bayes' rule

Bayes' rule updates a probability when new evidence arrives:

$$P(A | B) = P(B | A) \times P(A) \div P(B)$$

In English: *the probability of the cause given the evidence equals the probability of the evidence given the cause, times how common the cause is, divided by how common the evidence is.*

A worked Riverstone example

Riverstone's quality team tests moulded crates with a quick visual check. From history:

- **2%** of crates have a defect (the **prior**).
- The check flags **90%** of truly defective crates (**sensitivity**).
- It also flags **5%** of good crates (the **false-positive rate**).

A crate is flagged. What's the probability it's actually defective?

```
p_defect = 0.02
p_flag_given_defect = 0.90
p_flag_given_good = 0.05

p_flag = p_flag_given_defect * p_defect + p_flag_given_good * (1 - p_defect)
p_defect_given_flag = p_flag_given_defect * p_defect / p_flag

print(f"P(flag) = {p_flag:.4f}")
print(f"P(defect | flag) = {p_defect_given_flag:.4f} ({p_defect_given_flag*100:.1f}%)")
print(f"P(no defect | flag) = {1 - p_defect_given_flag:.4f}")
print(f"out of 10,000 crates: {10000*p_defect*p_flag_given_defect:.0f} true flags, "
      f"{10000*(1-p_defect)*p_flag_given_good:.0f} false flags")
```

```
P(flag) = 0.0670
P(defect | flag) = 0.2687 (26.9%)
P(no defect | flag) = 0.7313
out of 10,000 crates: 180 true flags, 490 false flags
```

Most flagged crates are fine. That result surprises nearly everyone, and the counts at the end are the way to explain it to a manager: out of 10,000 crates, 180 defective ones are flagged and 490 good ones are flagged too, so a flag is right only 27% of the time. Nothing is wrong with the test; defects are rare, and 5% of a large number of good crates is bigger than 90% of a small number of bad ones.

The same arithmetic explains why a "95% accurate" fraud model can produce mostly false alarms, why rare-disease screening needs a second test, and why an alert on a rare event needs a much lower false-positive rate than people expect (Chapter 20, section 20.8).

What to do with it as an analyst:

- **Always ask for the base rate.** "How often does this happen anyway?" is the single most useful question in a meeting about a predictive test.
- **Express it as counts**, not probabilities, when explaining. "Out of 10,000: 180 and 490" persuades; "0.269" doesn't.
- **Chain it.** A second, independent check on the flagged crates raises the probability sharply, which is why two-stage screening is standard.

21.8 Sampling and the central limit theorem

You almost never measure everything. You measure a **sample** and reason about the **population**. The gap between them is **sampling error**, and it's not a mistake: it's the ordinary variation you get from asking 200 people instead of 200,000.

```
population = deliveries["order_value"]
rng = np.random.default_rng(21)

print(f"population mean: ₹{population.mean():.2f} (n = {len(population):,})")
for n in (30, 200, 1000):
    sample_means = [rng.choice(population, size=n, replace=False).mean() for _ in range(2000)]
    sample_means = pd.Series(sample_means)
    print(f"samples of {n:>4}: mean of means ₹{sample_means.mean():.0f}, "
          f"sd of means ₹{sample_means.std():.0f}, "
          f"90% of samples between ₹{sample_means.quantile(0.05):.0f} and ₹"
          f"{sample_means.quantile(0.95):.0f}")
```

```
population mean: ₹24,839.53 (n = 45,040)
samples of   30: mean of means ₹24,941, sd of means ₹3,245, 90% of samples between ₹19,952 and ₹30,512
samples of  200: mean of means ₹24,831, sd of means ₹1,327, 90% of samples between ₹22,715 and ₹27,054
samples of 1000: mean of means ₹24,842, sd of means ₹572, 90% of samples between ₹23,950 and ₹25,815
```

Three things that output shows, and they're the whole of sampling theory in practice:

1. **Sample means cluster around the true mean.** The average of many sample means is the population mean: the sample mean is an **unbiased** estimate.
2. **Bigger samples are tighter.** The spread of sample means (the **standard error**) shrinks with the square root of n : to halve it, you need four times the data. That's why going from 200 to 1,000 helps a lot and from 10,000 to 50,000 rarely does.
3. **The distribution of sample means is roughly normal**, even though order values are strongly right-skewed. That's the **central limit theorem**, and it's why normal-based methods work on business data that is nothing like normal.

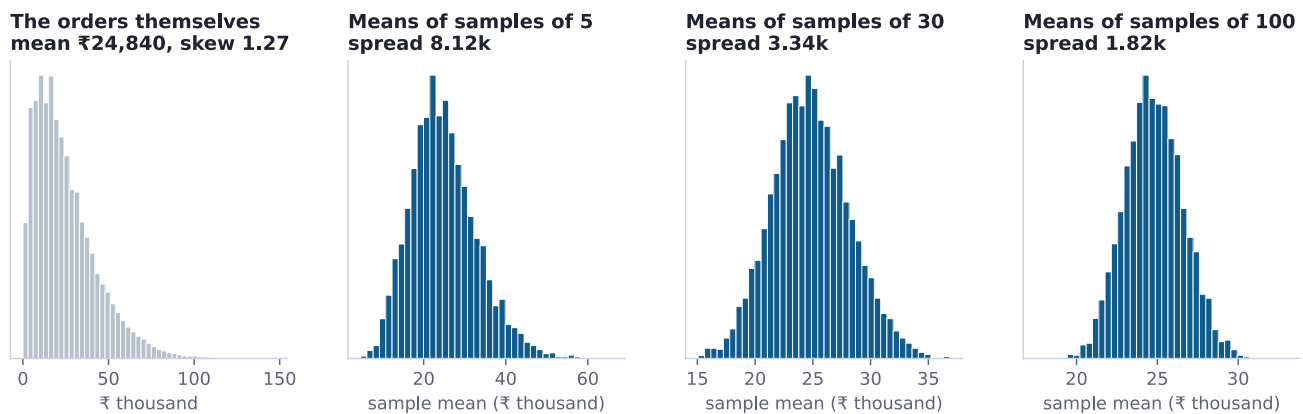


Figure 21.4 — The data is skewed; the averages are not. Larger samples give narrower, more bell-shaped distributions of the mean.

The standard error

```
n = 200
standard_error = population.std() / np.sqrt(n)
print(f"population sd: ₹{population.std():.2f}")
print(f"standard error of the mean for n = {n}: ₹{standard_error:.2f}")
print(f"roughly 95% of sample means fall within ±₹{1.96*standard_error:.0f} of the true mean")
```

```
population sd: ₹18,220.98
standard error of the mean for n = 200: ₹1,288.42
roughly 95% of sample means fall within ±₹2,525 of the true mean
```

The standard error is the standard deviation of the sample mean, and it's the bridge to Chapter 22's confidence intervals: a sample of 200 Riverstone orders puts the mean within about $\pm ₹2,500$ of the truth, 95% of the time. Quoting a sample mean without that range is how "our average order value rose to ₹25,100" becomes a decision it can't support.

Sampling in practice

- **Random means random.** "The first 200 rows" is not a sample; it's the oldest 200 orders, or one branch's file. Use `df.sample(n=200, random_state=42)`.
- **Stratify when groups differ.** If Kolkata is 12% of orders and you need branch-level answers, sample within each branch rather than hoping.
- **Beware survivorship and self-selection.** A satisfaction survey answered by 3% of customers measures the 3% who answer surveys.
- **Record the seed and the date.** A sample you can't reproduce is a number nobody can check.

21.9 Putting it together: profiling a measure

A repeatable routine for describing any numeric column, which is the project in miniature:

```
def profile(series, name):
    """Describe a numeric column the way a report should: centre, spread, shape, extremes."""
    s = series.dropna()
    q1, q3 = s.quantile(0.25), s.quantile(0.75)
    return {
        "measure": name, "n": len(s),
        "mean": round(s.mean(), 2), "median": round(s.median(), 2),
        "sd": round(s.std(), 2), "iqr": round(q3 - q1, 2),
        "p90": round(s.quantile(0.90), 2), "p99": round(s.quantile(0.99), 2),
        "min": round(s.min(), 2), "max": round(s.max(), 2),
        "skew": round(s.skew(), 2),
        "cv": round(s.std() / s.mean(), 2),
    }

profiles = pd.DataFrame([profile(deliveries["order_value"], "order value (₹)"),
                        profile(deliveries["delivery_days"], "delivery days"),
                        profile(deliveries[deliveries["branch"] == "Kolkata"]["delivery_days"],
                              "delivery days, Kolkata")])
print(profiles.to_string(index=False))
```

	measure	n	mean	median	sd	iqr	p90	p99	min	max
skew	cv									
order value (₹)	1.27	45040	24839.53	20700.0	18220.98	23135.25	49800.0	82790.08	546.25	143175.0
delivery days	0.73	45040	4.37	3.8	2.28	2.10	6.8	13.10	1.00	37.9
delivery days, Kolkata	3.13	5626	6.33	5.7	3.11	3.50	10.2	17.55	1.00	29.2
	0.52									
	1.70									
	0.49									

Then say it in sentences, because a table is not a finding:

Riverstone delivered 45,040 orders in 2025. A typical order was ₹20,700 (median), though the mean of ₹24,840 is higher because a minority of large orders pull it up; 10% of orders exceeded ₹49,800. Delivery took a median of 3.8 days and a mean of 4.4, with a long tail: 5% of orders took more than 8.2 days, and the slowest took 38. Kolkata is the outlier branch, with a median of 5.7 days, an interquartile range 2.5 times Mumbai's, and only 68% of orders inside its 7-day promise.

That paragraph is what statistics is for: six numbers, chosen deliberately, that let someone act.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Reporting the mean of skewed data alone	"Typical" order value nobody recognizes	Median alongside, and a histogram
Averaging averages	The company figure doesn't match the branches	Weight by count, or compute from raw rows
Quoting a centre with no spread	A promise that's broken half the time	Add IQR or a percentile
Using standard deviation on skewed data	"Mean $\pm$ sd" goes below zero	IQR and percentiles

Mistake	Symptom	Fix
Confusing sample and population sd	Small differences in small samples	<code>ddof=1</code> unless you have everything
Treating percentiles as interchangeable	Two tools, two answers	State the method (<code>PERCENTILE.INC</code> , <code>PERCENTILE.CONT</code>)
Deleting outliers to "clean" the data	The biggest customers vanish	Investigate; Chapter 14 decides, not the rule
Assuming normality	z-scores flag the wrong rows	Check skew; use distribution-free rules
Poisson for clustered events	Far more extreme weeks than predicted	Check whether variance $\approx$ mean
Confusing $P(A B)$ with $P(B A)$	"A third of late orders are Kolkata's" when the data says the reverse	Write both, with counts
Ignoring the base rate	A "95% accurate" test that's mostly false alarms	Bayes, expressed as counts out of 10,000
Treating sampling error as error	Panic at a 2% move in a survey	Standard error, and Chapter 22's intervals
Non-random samples	"The first 200 rows"	<code>sample(random_state=...)</code> , stratify if needed
Reporting more precision than the data supports	₹24,839.53 from a sample of 30	Round to what the standard error justifies

In the real world: the seven-day promise

Riverstone's sales team wanted a single delivery promise for the website: "Delivered in 5 days." The operations head objected on the grounds that Kolkata could not do it. The meeting went round twice on averages: sales quoting a company mean of 4.4 days, operations quoting the days they remembered.

Meera profiled it instead, and brought three numbers per branch: the median, the 95th percentile, and the share inside the current promise.

Branch	Median	95th percentile	Current promise	Inside it
Mumbai HO	3.2 days	5.7 days	5 days	90.8%
Bengaluru	3.8 days	6.9 days	5 days	78.0%
Delhi	4.4 days	8.4 days	6 days	80.0%
Kolkata	5.7 days	12.0 days	7 days	68.1%

Three things became obvious in about a minute:

1. **A single five-day promise would be kept for Mumbai and broken for everyone else.** The company average was never the relevant number; the branch distributions were.
2. **The right unit for a promise is a percentile, not an average.** "95% of orders within X days" is checkable; "average 4.4 days" tells a customer nothing about their order.

3. **Kolkata's problem was variability more than speed.** Its median was 2.5 days behind Mumbai's, but its interquartile range was 2.5 times as wide, and its 95th percentile was more than twice Mumbai's. Fixing the average would mean nothing; narrowing the spread would mean everything.

The team settled on branch-level promises at the 90th percentile, rounded up, and a monthly report of the share inside promise rather than the average. Operations opened an investigation into the Kolkata tail, which turned out to be dominated by consignments routed through a single transporter during the festive weeks.

Six months later the tail had shortened and the promise held at 92% company-wide. The statistics didn't fix anything; they changed what the meeting was about, from whose memory was right to which part of the distribution needed work.

Tools

- **Python 3.13 or 3.14** with `pandas`, `numpy`, `scipy`, `matplotlib`. Run here on Python 3.12, `pandas` 3.0.2, `numpy` 2.4.4, `scipy` 1.17.1, `matplotlib` 3.10.8.
- **Excel equivalents:** `AVERAGE`, `MEDIAN`, `MODE.SNGL`, `STDEV.S` / `STDEV.P`, `VAR.S`, `QUARTILE.INC`, `PERCENTILE.INC` / `.EXC`, `SKEW`, `KURT`, `NORM.DIST`, `NORM.INV`, `BINOM.DIST`, `POISSON.DIST`, `RAND`, `RANDBETWEEN`, and the Analysis ToolPak's Descriptive Statistics and Random Number Generation.
- **SQL:** `AVG`, `STDDEV_SAMP`, `PERCENTILE_CONT`, `PERCENTILE_DISC`, `NTILE` (Chapter 13).
- **Companion files (`companion/ch21/`):** `build_ch21_files.py` and `delivery_times_2025.csv` (45,040 delivered 2025 orders with order value, branch, promised days, actual delivery days, and an on-time flag). The delivery times are generated from a documented seeded model, because Riverstone's ERP data has no delivery dates; everything else comes from the full dataset.

The project: profile Riverstone's order values and delivery times

Goal: a one-page statistical profile that changes what a meeting argues about.

Option A: your own data. Any measure you report regularly: ticket resolution time, days sales outstanding, machine downtime, basket size.

Option B: Riverstone. Use `delivery_times_2025.csv` and the full dataset.

Steps

1. **Profile both measures** with the `profile()` function from section 21.9: `n`, mean, median, `sd`, `IQR`, `p90`, `p99`, min, max, skew, CV.
2. **Draw the distributions:** a histogram with sensible bins (Chapter 15) and a box plot per branch, with the promise marked.
3. **Describe the shape in words.** Is it skewed? How heavy is the tail? What does the gap between mean and median mean for anyone setting a target?

4. **Quantify the tail:** what share of orders exceed the promise, and how much revenue sits behind them?
5. **Set a defensible promise:** for each branch, the 90th percentile rounded up, and the share of orders that would meet it. Compare with the current promise.
6. **Probability questions:** given a day with 40 deliveries at each branch's late rate, what's the chance of more than 10 late? (Binomial.) If complaints average 9.4 a week, how often would you see 15 or more? (Poisson.)
7. **A Bayes question:** the branch flags orders it expects to be late, catching 80% of truly late orders and flagging 10% of on-time ones. If an order is flagged, what's the probability it's actually late? Express it as counts out of 10,000.
8. **Sampling:** take random samples of 30, 200, and 1,000 orders; report the sample mean and its standard error, and state how far a sample of 200 can be from the truth.
9. **Write the page:** six to eight sentences, three numbers per branch, one chart, and one recommendation.

What good looks like: the profile matches section 21.9 (order values: median ₹20,700, mean ₹24,840, skew 1.27; delivery days: median 3.8, mean 4.37, p95 8.2, skew 3.13), the branch table matches the story, and the recommendation is stated as a percentile promise rather than an average.

Stretch goals

- Split delivery times by month and test whether the festive season (October and November) is slower, and by how much.
- Fit a lognormal distribution to delivery days and compare its 95th percentile with the empirical one.
- Work out the sample size needed to estimate the mean order value within $\pm ₹500$ with 95% confidence (Chapter 22 gives the formula; the standard error in section 21.8 gives you enough to try).
- Repeat the whole profile for order values by segment, and say which segment's promise would be hardest to keep.

Timed challenge: forty minutes

Use `delivery_times_2025.csv`. Answers at the end of the chapter.

- **Level 1:** Mean, median, and mode of order value; mean and median of delivery days.
- **Level 2:** Standard deviation and IQR of delivery days, company-wide.
- **Level 3:** The 90th and 99th percentiles of delivery days, and the slowest single delivery.
- **Level 4:** Median, IQR, and 95th percentile of delivery days for each branch.
- **Level 5:** The share of orders inside the promise, company-wide and per branch.
- **Level 6:** Using the IQR rule, how many orders are outliers on delivery days, and what share is that?
- **Level 7:** $P(\text{late})$, $P(\text{late} \mid \text{Kolkata})$, and $P(\text{Kolkata} \mid \text{late})$.

- **Bonus:** Take 2,000 random samples of 200 orders; report the standard deviation of the sample means, and compare it with the standard error formula.
-

You've got it when...

- ☐ You choose between mean, median, and mode for a reason you can state.
 - ☐ You never quote a centre without a spread.
 - ☐ You use IQR and percentiles on skewed data, and standard deviation where it's meaningful.
 - ☐ You can explain why the mean sits above the median in most money data.
 - ☐ You write service levels as percentiles.
 - ☐ You can name the distribution a measure is likely to follow and what that implies.
 - ☐ You can compute a binomial or Poisson probability to set a sensible threshold.
 - ☐ You never confuse $P(A | B)$ with $P(B | A)$, and you ask for the base rate.
 - ☐ You can explain a Bayes result as counts out of 10,000.
 - ☐ You know that sample means cluster normally around the truth, that the spread shrinks with the square root of n , and roughly how far a sample of 200 can be off.
 - ☐ You profile a new measure the same way every time, and turn the table into sentences.
-

Recap

- **Centre:** mean uses everything and is dragged by extremes; median is the middle and isn't; mode is for categories. On Riverstone's orders the mean is ₹24,840 and the median ₹20,700, and the gap is the story.
- **Spread:** range is fragile, variance is in squared units, standard deviation is the default for symmetric data, IQR is the honest choice for skewed data. Two branches with similar averages can have very different experiences.
- **Percentiles** describe what customers actually get, which is why service levels use them: 95% of Riverstone's orders arrive within 8.2 days.
- **Shape:** positive skew is the norm for money and durations. Compare mean with median as a quick test. Outlier rules (IQR, z-score) produce candidates, not verdicts.
- **Distributions:** normal for many small influences, binomial for counts of yes/no trials, Poisson for event counts at a steady rate, uniform for equal likelihood. Each answers "how unusual is this?" with a formula.
- **Probability rules:** complement, addition (subtract the overlap), multiplication (with conditioning), and conditional probability. $P(A | B) \neq P(B | A)$.
- **Bayes' rule** updates a prior with evidence, and the base rate usually dominates: with a 2% defect rate and a 5% false-positive rate, most flagged crates are fine.

- **Sampling:** sample means are unbiased, their spread is the standard error, it shrinks with $\sqrt{n}$, and the central limit theorem makes them roughly normal even when the data is not.
 - **A profile** is n, mean, median, sd, IQR, p90, p99, min, max, skew, CV — and then two or three sentences that tell someone what to do.
-

Practice exercises

Use `companion/ch21/delivery_times_2025.csv` and the full dataset.

Warm-up

1. For each, say whether you'd report the mean, the median, or the mode, and why: order value; delivery days; the most common product in an order; total revenue per month; salary in a small team including the owner.
2. A branch's four monthly averages are 3.1, 3.4, 3.0, and 6.2 days. What's suspicious about reporting 3.9 as the year's average?
3. Explain variance and standard deviation to a sales manager in two sentences, without the word "squared" in the second one.
4. Riverstone's mean order value is ₹24,840 and the median ₹20,700. What does that tell you about the shape, and what would it look like if the two were equal?
5. Why is "average delivery 4.4 days" a bad promise, and what would you write instead?
6. What's the difference between $P(\text{late} \mid \text{Kolkata})$ and $P(\text{Kolkata} \mid \text{late})$? Give a sentence for each using the real numbers.

Core

7. Compute the mean, median, mode, standard deviation, and IQR of order value. Which two would you put in a management pack, and why?
8. Compute the same statistics for delivery days per branch, and rank the branches by median and by IQR. Does the ranking change?
9. What is the 90th percentile of delivery days for each branch? If each promise were set there and rounded up, what would the promises be?
10. What share of orders would meet those new promises? Compare with the current 81.8%.
11. Using the IQR rule on delivery days, how many outliers are there, and which branch has the most as a share of its orders?
12. Compute the skew of order value and of delivery days. Which is more skewed, and what does that imply for reporting an average?
13. What share of 2025 revenue sits in orders that were delivered late?
14. With 40 deliveries in a day and Kolkata's late rate, what's the probability that more than 15 are late? (Binomial.)
15. If complaints average 9.4 a week, what's the probability of a week with 15 or more? Of a week with none?

16. A crate's weight is normal with mean 500 g and standard deviation 4 g. What share falls outside 492–508 g? What limit would only 1 in 1,000 crates fall below?
17. Compute $P(\text{late})$, $P(\text{late} \mid \text{festive months})$, and $P(\text{festive} \mid \text{late})$ using October and November as the festive season. Are lateness and season independent?
18. The branch flags orders it expects to be late: it catches 80% of late orders and flags 10% of on-time ones. Given the real late rate, what's $P(\text{late} \mid \text{flagged})$? Express it as counts out of 10,000.
19. Take 2,000 samples of 200 orders and plot the distribution of the sample means. What is its standard deviation, and how close is it to the standard-error formula?
20. How large a sample would you need for the standard error of the mean order value to be under ₹500?
21. Write the `profile()` function from section 21.9 and run it on order value, delivery days, and delivery days for each branch. Present it as a table a manager could read.
22. Turn that table into four sentences with a recommendation.

Stretch

23. Split delivery days by month. How much slower are October and November than the rest of the year, at the median and at the 95th percentile?
24. Fit a lognormal distribution to delivery days (`scipy.stats.lognorm.fit`) and compare its 90th and 99th percentiles with the empirical ones. Where does the fit fail?
25. Simulate a day of 40 deliveries 10,000 times using each branch's late rate, and report how often more than a quarter are late. Compare with the binomial answer.
26. Using the delivery data, estimate the probability that a customer's next two orders are both late, first assuming independence and then conditioning on branch. Which is bigger, and why does that matter for a customer-experience metric?

Think about it

27. A manager says "our average delivery time improved from 4.6 to 4.4 days this month; the process is working". What three questions do you ask?
28. When is it legitimate to remove an outlier from a reported statistic, and what must you disclose?
29. A dashboard shows a 95% "on-time" figure; a customer survey says a third of customers experienced a late delivery in the last year. Can both be true?

Key terms

descriptive statistics · mean · median · mode · weighted average · average of averages · spread · range · variance · standard deviation · sample versus population (ddof) · interquartile range · quartile · percentile · interpolation method · service level · skew · kurtosis · tail · outlier · IQR rule · z-score · coefficient of variation · distribution · probability density · probability mass · normal distribution · 68–95–99.7 rule · binomial distribution · Poisson distribution · uniform distribution · rate (λ) · probability · complement rule · addition rule · multiplication rule · conditional probability · independence · base

rate · base rate fallacy · Bayes' rule · prior · likelihood · posterior · sensitivity · false-positive rate · population · sample · random sample · stratified sample · sampling error · unbiased estimate · standard error · central limit theorem · seed

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 22, Statistics Without Fooling Yourself:** confidence intervals, hypothesis tests, p-values, A/B tests, and the traps around all of them.
- **Chapter 15** already used these ideas visually: histograms, box plots, and the quartiles behind them.
- **Chapter 20:** thresholds and alerts that are set from a distribution rather than a round number.
- **Chapter 24, Forecasting:** trend, seasonality, and prediction intervals, which are standard errors in a different coat.
- **Part IV:** every model assumes distributions, sampling, and the difference between training data and the world.
- **Interview preparation:** the Statistics & Analytics Question Bank (Chapter 73) covers exactly this ground, including Bayes and the central limit theorem.

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. Order value: **median** (skewed), with the mean if totals matter. Delivery days: **median** plus a percentile. Most common product: **mode**. Total revenue per month: neither — it's a total, and you'd compare it with last year. Small team's salary including the owner: **median**, because one salary dominates the mean.
2. 3.9 is the average of four monthly averages, so it ignores how many orders each month had, and it hides that one month (6.2) was very different. Report the median with the range, or compute from the orders, and show the monthly series.
3. "Variance measures how far orders sit from the average, on average. The standard deviation puts that back into days, so you can say most orders land within about so many days either side."
4. A long right tail: a minority of large orders. If they were equal, the distribution would be roughly symmetric, and the mean would describe a typical order.
5. Because half the orders are slower than the mean, and a fifth are much slower. Write "**90% of orders arrive within X days**", per branch.
6. $P(\text{late} \mid \text{Kolkata}) = 31.9\%$: about a third of Kolkata's orders arrive late. $P(\text{Kolkata} \mid \text{late}) = 21.8\%$: about a fifth of the company's late orders come from Kolkata. The first is about Kolkata's process; the second is about where the lateness sits.

7. Mean ₹24,839.53, median ₹20,700.00, mode ₹1,725.00, sd ₹18,220.98, IQR ₹23,135.25. Put the **median** (typical order) and the **mean** (consistent with revenue) in the pack, and note the skew.

8. By median: Mumbai HO 3.2, Bengaluru 3.8, Delhi 4.4, Kolkata 5.7. By IQR: Mumbai HO 1.4, Bengaluru 1.9, Delhi 2.2, Kolkata 3.5. The ranking is the same here, which is itself worth saying: Kolkata is both slower and less predictable.

9. Branch 90th percentiles are 5.0 (Mumbai HO), 6.0 (Bengaluru), 7.2 (Delhi) and 10.2 (Kolkata) days, so rounded-up promises would be 5, 6, 8, and 11 days.

10. By construction about 90% per branch, which is higher than today's 90.8 / 78.0 / 80.0 / 68.1. The honest framing: the promise gets longer for three branches, and the share kept rises to a consistent 90%.

11. 2,220 orders (4.9%) are above the company-wide fence of 8.25 days. Kolkata contributes the largest share of its own orders, which is what the fat tail in Figure 21.2 shows.

12. Delivery days (skew 3.13) is far more skewed than order value (1.27). The more skewed the measure, the more misleading a bare average is, and the more important percentiles become.

13. Compute `deliveries.loc[~deliveries["on_time"], "order_value"].sum() / deliveries["order_value"].sum()` : **19.9%** of delivered revenue arrived late, which is the number that makes the operations case in money rather than percentages.

14. With $n = 40$ and $p = 0.319$, $1 - \text{stats.binom.cdf}(15, 40, 0.319) \approx \mathbf{0.18}$: nearly one day in five. An alert set at "more than 15 late" would fire about once a week in Kolkata, which is too often to be useful.

15. $P(15 \text{ or more}) \approx 0.0559$; $P(\text{none}) \approx 0.00008$. A complaint-free week would be a real surprise.

16. Outside 492–508 g is ± 2 sd, so about **4.6%**. The 1-in-1,000 lower limit is `stats.norm.ppf(0.001, 500, 4)` $\approx \mathbf{487.6 \text{ g}}$.

17. Using October and November as festive: lateness is higher in the festive months, so $P(\text{late} \mid \text{festive}) > P(\text{late})$, and the two are not independent — which is exactly why a company-wide annual late rate is a poor baseline for a festive-season alert.

18. With a late rate of 18.2%: out of 10,000 orders, 1,820 are late and 8,180 are not; the flag catches 1,456 late ones and 818 on-time ones, so $P(\text{late} \mid \text{flagged}) = 1,456 \div 2,274 \approx \mathbf{64\%}$. Better than the 27% in the crate example, because lateness is far more common than defects: the base rate does the work.

19. The standard deviation of the 2,000 sample means comes out around **₹1,270–1,290** depending on the seed, and the formula `population sd /  $\sqrt{200}$` gives ₹1,288. They agree, which is the central limit theorem doing its job.

20. $n \geq (\text{sd} / 500)^2 = (18,221 \div 500)^2 \approx \mathbf{1,329}$ orders.

21. See section 21.9. Present it with the measure in the first column and round sensibly: nobody needs `order_value` to the paise in a management pack.

22. For example: "A typical order is ₹20,700, but the mean of ₹24,840 shows a minority of large orders. Delivery takes a median of 3.8 days, with 5% of orders beyond 8.2. Kolkata is the exception on both speed and consistency: median 5.7 days and an interquartile range 2.5 times Mumbai's. Recommendation: set branch-level promises at the 90th percentile and report the share inside promise, not the average."

23. October and November are slower at every point of the distribution, and much slower in the tail: the festive penalty added to lead times widens the right tail more than it moves the median. Report both, because the tail is what customers complain about.
24. A lognormal fits the body of the delivery-time distribution well and typically understates the extreme tail, because the real data includes a small share of badly broken deliveries that aren't part of the same process. That mismatch is a finding: two processes, not one.
25. The simulation and the binomial agree closely, which is the point of the exercise: the binomial is the model of that simulation. Simulation earns its place when the situation is too complicated for a formula.
26. Assuming independence, $P(\text{both late}) = P(\text{late})^2 \approx 3.3\%$. Conditioning on branch, a Kolkata customer's two orders are each late about 31.9% of the time, so $\approx 10.2\%$. The conditional version is larger and more honest: lateness clusters by customer because customers belong to branches, and a customer-experience metric built on the independent assumption will understate how many customers see repeated failures.
27. (1) How much does delivery time vary month to month anyway — is 0.2 days inside the normal range? (2) Did the mix change: more Mumbai orders and fewer Kolkata ones would improve the average with no process change. (3) What happened at the 90th percentile, where customers feel it? A median and a percentile per branch answer all three.
28. When the value is a known error (Chapter 14), when it comes from a different process that you state clearly ("excluding the two consignments lost in transit"), or when a rule agreed in advance excludes it. You must disclose the rule, the count removed, and the effect on the number, and you should show the figure both ways.
29. Yes. A 95% on-time rate measures **orders**; the survey measures **customers**, and a customer who places twenty orders a year has a much higher chance of seeing at least one late delivery: $1 - 0.95^{20} \approx 64\%$. Per-order and per-customer metrics answer different questions, and the customer's experience is usually the one that drives complaints.

Timed challenge answers. Level 1: order value mean ₹24,839.53, median ₹20,700.00, mode ₹1,725.00; delivery days mean 4.37, median 3.8. Level 2: sd 2.28 days, IQR 2.10 days (Q1 3.0, Q3 5.1). Level 3: p90 6.8 days, p99 13.1 days, slowest 37.9 days. Level 4: medians 3.2 / 3.8 / 4.4 / 5.7 and IQRs 1.4 / 1.9 / 2.2 / 3.5 for Mumbai HO, Bengaluru, Delhi, Kolkata; p95s 5.7 / 6.9 / 8.4 / 12.0. Level 5: 81.8% company-wide; Mumbai HO 90.8%, Delhi 80.0%, Bengaluru 78.0%, Kolkata 68.1%. Level 6: 2,220 orders, 4.9%. Level 7: $P(\text{late})$ 18.2%, $P(\text{late} \mid \text{Kolkata})$ 31.9%, $P(\text{Kolkata} \mid \text{late})$ 21.8%. Bonus: around ₹1,270–1,290 depending on the seed, against a formula value of ₹1,288.